Experiment No. 4

Edge Computing Lab	
Name of Student:	
Class :	**Date of Performance:**
Roll No :	**Date of Checking :**
	Sign :

Mitigating Resource Exhaustion (DoS) Attacks via Container Isolation

Objective:

To demonstrate how a compromised or malicious microservice can crash a resource-constrained Edge device (Raspberry Pi) and how to implement Container Isolation policies (CPU/Memory limits) to neutralize the threat.

An attacker has injected a malicious script into one of your edge containers (or a developer wrote bad code with an infinite loop). This script tries to consume 100% of the Raspberry Pi's CPU and RAM. You must observe the failure and then apply security policies to contain it.

Tasks:

- The "Malicious" Payload (The Stress Test)
- The Attack (Simulating Failure)
- The Countermeasure (Implementing Isolation)
- Verification & Trust

Theory:-

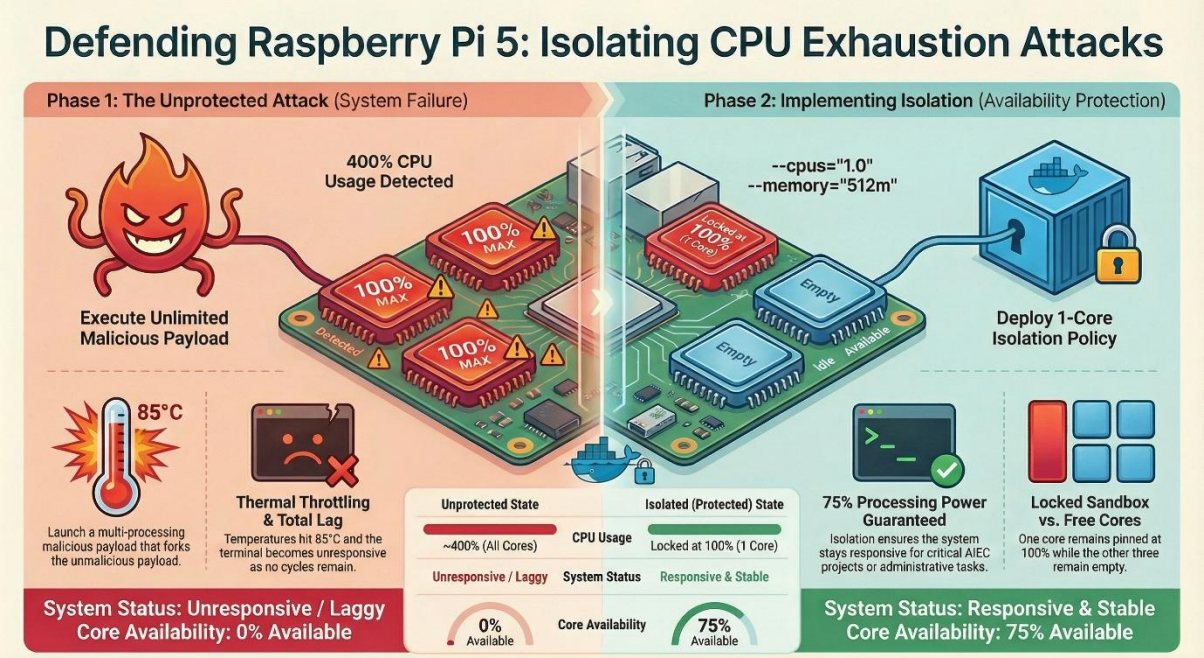
This lab experiment is on a Raspberry Pi 5 to demonstrate how container isolation prevents a complete system shutdown during a CPU exhaustion attack. The process begins by deploying a malicious Python script within a Docker container designed to hijack all available processing cores, leading to thermal throttling and extreme interface lag. By monitoring system metrics like temperature and core utility, the author illustrates the catastrophic impact of an unregulated process.

In an unprotected state, simultaneous heat generation from all four cores can cause rapid thermal throttling (temperatures rising to 80°C - 85°C) and system unresponsiveness. Under isolation, while the single active core still generates heat, the temperature rise is slower, and the system remains responsive because the other cores are idle

To combat this, the tutorial introduces resource limits, such as capping the attacker to a single core and a specific memory threshold. By "sacrificing" one core to the intensive or malicious process, the policy guarantees that the remaining **75% of the Raspberry Pi 5's processing power stays free**. This ensures resources are available for critical work or administrative tasks, preventing a System Denial of Service (DoS).

This strategic sandboxing ensures that even while a malicious process runs at maximum intensity, most of the processor's power remains available for other tasks. Ultimately, the source highlights that security policies are essential for maintaining device availability and responsiveness in edge computing environments

Block Diagram:-



1. The Malicious Code (stress_test.py)

This module is designed to identify the total number of CPU cores and launch an infinite loop on each one to saturate the ARM architecture.

Python

```python
# -*- coding: utf-8 -*-
import multiprocessing
import os
import time

def cpu_exhaustion():
    # Force the CPU core into an infinite calculation loop
    while True:
        pass

if __name__ == "__main__":
    cores = multiprocessing.cpu_count()
    print(f"Malicious Payload Active. Detected {cores} cores.")
    # Target every available core to trigger a System DoS
    for _ in range(cores):
        p = multiprocessing.Process(target=cpu_exhaustion)
        p.daemon = True
        p.start()

    # Keep the master process alive
    while True:
        time.sleep(1)
```

Justification: By using multiprocessing, the script ensures it doesn't just hang one core, but instead tries to hijack the entire physical processor.

---

2. Container Creation (Dockerfile.malicious)

We use a lightweight Python environment to ensure the attack is efficient and has minimal overhead.

```
Dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY stress_test.py .
# Disable buffering so we see the "Payload Active" message immediately
ENV PYTHONUNBUFFERED=1
CMD ["python", "stress_test.py"]
```

Justification: The PYTHONUNBUFFERED flag is critical for Edge devices so that logs are written to the terminal in real-time without being stuck in memory.

3. Executing the "Unprotected" Attack

First, we build the image and run it without any safety limits to observe the failure.

Bash

Build the image

```
docker build -f Dockerfile.malicious -t malicious-app .
```

Start the attack without limits

```
docker run -d --name attacker-unlimited malicious-app
```

4. Verification (The "Failure" State)

While the attack is running, check these three metrics to see how the Raspberry Pi 5 reacts.

Check Container Stats

Bash

```
docker stats attacker-unlimited
```

Justification: You will see **CPU %** near **400%**. This confirms the container has bypassed "fair use" and is consuming the entire device's power.

Check Temperature Rise

Bash

```
vcgencmd measure_temp
```

Justification: Without limits, all 4 cores generate heat simultaneously. You will likely see the temperature climb rapidly toward **80°C - 85°C**, leading to thermal throttling.

Check CPU Core Status

Bash

```
htop
```

Justification: In the htop visualizer, all four green bars will be pinned at 100%. This explains why the terminal feels "laggy"—there are no CPU cycles left for your commands.

5. Implementing Isolation (The Countermeasure)

Now we apply the security policy. We kill the runaway container and restart it with a **1-core cap**.

Bash

```
# Forcefully kill the unlimited attacker
docker kill attacker-unlimited && docker rm attacker-unlimited
```

Deploy with 1-Core Isolation and 512MB RAM limit

```
docker run -d \
  --name attacker-isolated \
  --cpus="1.0" \
  --memory="512m" \
  malicious-app
```

---

6. Post-Isolation Verification/ Expected Results

Check the metrics again to see the "Sandbox" effect.

- **Stats:** docker stats will now show the CPU locked at **100%** (exactly 1 core), even though the script is trying to use 400%.
- **Core Status:** htop will show **one core at 100%** and the **other three cores nearly empty**.
- **Temperature:** Run vcgencmd measure_temp. The temp may still be high (e.g., **84°C**) because one core at 100% still generates heat, but the rise is slower and the system stays responsive.

Question and Answers to write by Students:

Q1: What specific type of cyberattack was simulated in this lab, and how does it affect an Edge device?

Answer: The experiment simulated a **Local Denial of Service (DoS) attack via Resource Exhaustion**. In an Edge environment, this attack occurs when a compromised or poorly written container attempts to hijack the hardware's limited resources.

- **Mechanism:** By spawning multiple infinite loops, the script saturates the **CPU cycles**.
- **Thermal Impact:** The intense calculation generates heat, pushing the Raspberry Pi 5 toward its thermal limit (80°C+), which triggers **thermal throttling**.

- **System Impact:** Without isolation, the attack consumes all 4 cores of the Pi's ARM architecture, "starving" the Operating System and preventing it from responding to legitimate user commands or safety-critical tasks.

Q2: Why did the CPU usage show as ~400% in the docker stats output during the unprotected attack?

Answer: Docker calculates CPU percentage relative to a **single core**.

- **Multi-core Architecture:** The Raspberry Pi 5 features a quad-core processor.
- **Calculation:** Each core represents 100% capacity. Therefore, a reading of **394.93%** indicates that the malicious script successfully scaled across nearly the entire physical processor.
- **Process Multiplication:** This was achieved because the stress_test.py script used the multiprocessing module to detect the total core count and launch a dedicated "worker" process for every core available on the device.

Q3: What is "Container Isolation," and how do Cgroups facilitate this as a countermeasure?

Answer: Container Isolation is a security policy that "sandboxes" a container's access to physical hardware.

- **Cgroups (Control Groups):** This is the underlying Linux kernel feature that Docker uses to enforce limits.
- **The Policy:** By running the container with the --cpus="1.0" flag, the kernel is instructed to only provide that specific container with enough CPU cycles to equal one core.
- **The Result:** Even if the malicious code tries to run 4 loops, the kernel "throttles" the container's total consumption to 100%, effectively protecting the remaining **3 CPU cores** so the rest of the system stays operational.

Q4: Even after implementing a 1-core CPU limit, the temperature remained at 84°C. Does this mean the isolation failed?

Answer: No, the isolation was a **success**. The goal of resource isolation in this context is **Availability**, not necessarily cooling.

- **Responsiveness:** While 84°C is high, the Pi remained responsive because 3 cores were free to handle the OS and terminal commands.
- **Thermal Density:** On a compact board like the Pi 5, a single core running at 100% generates significant heat that "soaks" into the neighboring silicon.

- **Safety Threshold:** 84°C is the point where the Pi 5 begins **thermal throttling**. Isolation ensures that only the *attacker* gets throttled, whereas an unlimited attack would have throttled the *entire* system, causing a total blackout of all services.

Q5: How does this lab relate to real-world Edge AI projects, such as a "Seizure Prediction Headband"?

Answer: This lab demonstrates the importance of **Critical Service Preservation**.

- **Resource Competition:** In a project like your **Seizure Prediction Headband**, multiple services might run at once: one for sensor data collection, one for AI inference (Edge-AI), and one for cloud logging.
- **Mitigating Failure:** If the "Cloud Logging" service becomes compromised or hits a bug and starts an infinite loop, **Container Isolation** ensures it cannot steal the CPU cycles needed by the "Seizure Prediction" algorithm.
- **Deployment Safety:** It proves that by using **Docker** and **Cgroups**, researchers can deploy complex AI models on resource-constrained hardware like the Raspberry Pi or Jetson Nano with a "safety net" that prevents one service from crashing the entire medical or industrial device.

Conclusion:-